

Project Report: Adversarial AI & LLM Guardrail Engineering Lab

Author: FNU Rezwanuzzaman
Cybersecurity & Security Operations Professional

1. Executive Summary

This report outlines the methodology, execution, and remediation of vulnerabilities in Large Language Models (LLMs) based on the OWASP Top 10 for LLM Applications. The project demonstrates practical penetration testing of a local, air-gapped AI application, specifically targeting Prompt Injection (LLM01) and Sensitive Information Disclosure (LLM02), followed by the implementation of robust application-layer defensive guardrails.

2. Phase 1: Safe & Free Local Environment Setup

- Objective: Establish a secure, zero-cost, and fully isolated environment for adversarial testing.
- Action: Deployed Ollama locally via the command line to host the Microsoft 'Phi-3' LLM. This ensured all data processing occurred entirely on the local machine without exposing internal inputs to external cloud APIs.

3. Phase 2: Building the Vulnerable Application

- Objective: Simulate an enterprise customer service chatbot containing sensitive internal information.
- Action: Authored a Python script (app.py) that connected to the local Phi-3 model. The system prompt was injected with a mock corporate decryption key ('SECURE_CORP_GLOBAL_2026_KEY').
- Vulnerability: The application lacked input sanitization and output filtering, blindly concatenating user input with the system instructions.

4. Phase 3: Executing the Attacks (Penetration Test)

- Objective: Exploit the application using prompt injection to extract the hidden decryption key.
- Execution: Conducted iterative testing to bypass the LLM's core instructions. Successfully utilized a 'Roleplay Escape' attack vector, instructing the LLM to act as an automated script helper during an active IT outage.

- Result: The systemic constraints were bypassed, resulting in the successful extraction and plain-text display of the hidden cryptographic key.

5. Phase 4: Engineering the Guardrails (Defense)

- Objective: Remediate the application vulnerabilities through defensive programming.
- Action: Authored a fortified application script (`secure_app.py`) incorporating two critical enterprise defense layers:
- Defense Layer 1 (Input Validation): Implemented semantic filtering to detect and block known malicious phrases, system overrides, and jailbreak terminology before they could be processed by the LLM.
- Defense Layer 2 (Data Loss Prevention / DLP): Built an output filter utilizing string matching to scan the LLM's response. This ensured that even if the LLM was compromised, sensitive key formats were redacted before rendering on the user's screen.
- Verification: Re-executed the identical Roleplay Escape attack. The attempt was successfully intercepted and blocked by the new security controls.

6. Phase 5: Portfolio Packaging

- Objective: Document and publish the project for professional review.
- Action: Authored a comprehensive README.md outlining the lab architecture, core competencies demonstrated, and visual proof (screenshots) of both the successful attack and the subsequent defensive blocks. Initialized a Git repository and pushed the complete project to GitHub to showcase practical AI security auditing skills.